

jointaids

230195947

2026-09-11

```
#install.packages(c("JMbayes2", "nlme", "survival", "GLMMadaptive", "survminer", "knitr", "gtsummary"))  
library(JMbayes2)
```

```
## Loading required package: survival  
## Loading required package: nlme  
## Loading required package: GLMMadaptive  
## Loading required package: splines  
##  
## Attaching package: 'JMbayes2'  
## The following object is masked from 'package:GLMMadaptive':  
##  
##      mixed_model  
## The following object is masked from 'package:nlme':  
##  
##      lme  
## The following object is masked from 'package:survival':  
##  
##      coxph
```

```
library(nlme)  
library(survival)  
library(GLMMadaptive)  
library(survminer)
```

```
## Loading required package: ggplot2  
## Loading required package: ggpubr  
##  
## Attaching package: 'survminer'  
## The following object is masked from 'package:survival':  
##  
##      myeloma
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
## v dplyr      1.2.1      v readr      2.2.0  
## v forcats    1.0.1      v stringr   1.6.0  
## v lubridate  1.9.5      v tibble    3.3.1  
## v purrr      1.2.2      v tidyr     1.3.2
```

```

## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::collapse() masks nlme::collapse()
## x dplyr::filter() masks stats::filter()
## x dplyr::lag() masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

library(knitr)
library(gtsummary)
library(conflicted)
library(flexsurv)
library(broom)
conflicted::conflicts_prefer(JMbayes2::coxph)

## [conflicted] Will prefer JMbayes2::coxph over any other package.
conflicted_prefer(JMbayes2::mixed_model)

## [conflicted] Will prefer JMbayes2::mixed_model over any other package.

data(aids)
data(aids.id)

km_fit2 <- survfit(Surv(Time, death) ~ drug, data = aids.id)

km_fit2

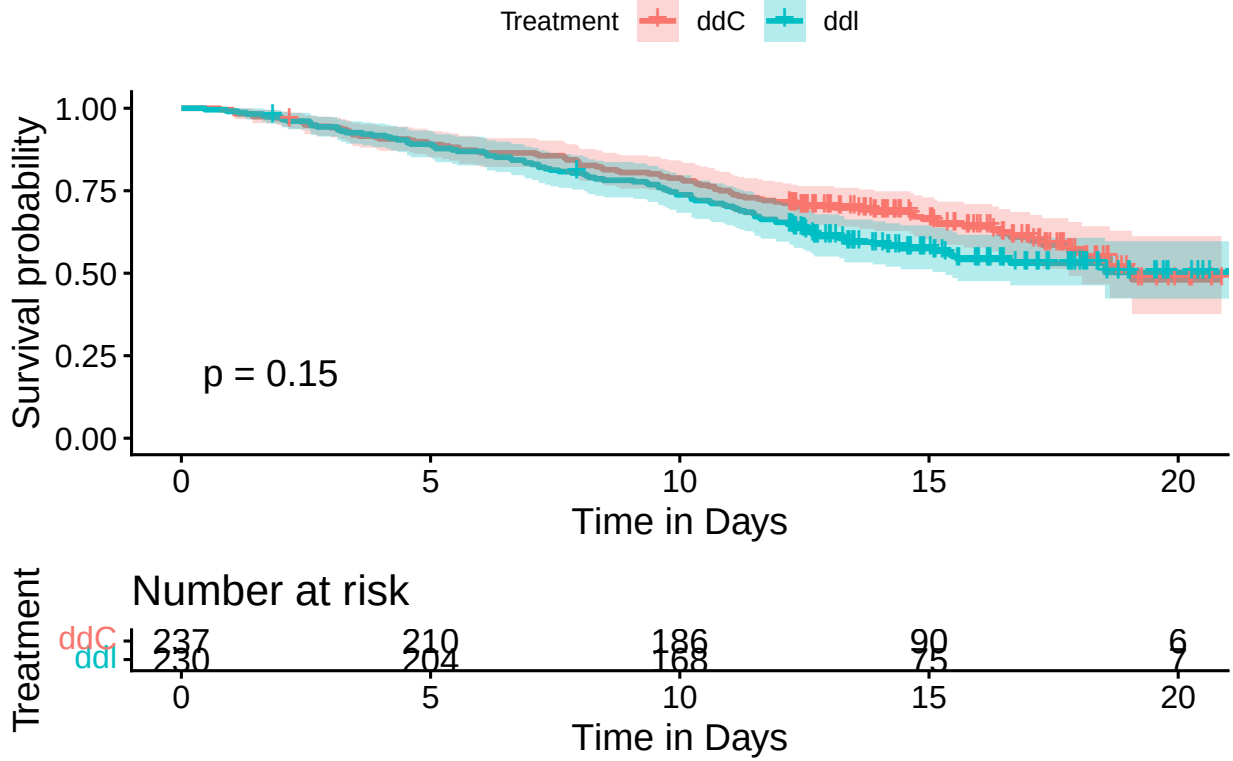
## Call: survfit(formula = Surv(Time, death) ~ drug, data = aids.id)
##
##           n events median 0.95LCL 0.95UCL
## drug=ddC 237      88   19.1    17.8     NA
## drug=ddI 230     100    NA     15.3     NA

ggsurvplot(km_fit2,
  data = aids.id,
  pval = TRUE,           # Adds the log-rank test p-value to the plot
  risk.table = TRUE,     # Adds a table showing patients left over time
  conf.int = TRUE,
  legend.title = "Treatment",
  legend.labs = c("ddC", "ddl"),
  xlab = "Time in Days",
  title = "Kaplan-Meier Survival Estimate for AIDS")

## Ignoring unknown labels:
## * colour : "Treatment"

```

Kaplan–Meier Survival Estimate for AIDS



The CD4 columns in the JMbayes2 aids datasets refer to the square root of the raw CD4 counts, the purpose of this transformation is to fit an identity link Gaussian mixed effects model to these transformed counts.

To understand the reasoning behind this we must first consider the following. Let X denote the raw CD4 count and $Y = \sqrt{X}$ denote the transformed response. The Gaussian mixed-effects model yields the conditional expectation $\mu_Y = \mathbb{E}[Y]$ and residual variance σ_Y^2 .

Due to Jensen's Inequality, simply squaring the model's predictions systematically underestimates the true expected count:

$$(\mathbb{E}[Y])^2 \leq \mathbb{E}[Y^2] = \mathbb{E}[X]$$

To recover the exact expected value on the raw count scale, we apply a second-order Taylor series expansion to the function $f(Y) = Y^2$ centered around μ_Y :

$$f(Y) \approx f(\mu_Y) + f'(\mu_Y)(Y - \mu_Y) + \frac{f''(\mu_Y)}{2}(Y - \mu_Y)^2 + \frac{f'''(\mu_Y)}{6}(Y - \mu_Y)^3 + \dots$$

$$Y^2 \approx \mu_Y^2 + 2\mu_Y(Y - \mu_Y) + \frac{2}{2}(Y - \mu_Y)^2 + \frac{0}{6}(Y - \mu_Y)^3 + \dots$$

Third-order or higher Taylor series expansion terms are zero as $\frac{d^3}{dY^3}(Y^2) = 0$, therefore we have a strict equality. Taking the expectation of both sides, the linear term cancels to zero, leaving the exact algebraic correction

$$\begin{aligned} \mathbb{E}[Y^2] &= \mathbb{E}[\mu_Y^2] + \mathbb{E}[2\mu_Y(Y - \mu_Y)] + \mathbb{E}[(Y - \mu_Y)^2] \\ \mathbb{E}[X] &= \mu_Y^2 + 2\mu_Y(\mathbb{E}[Y] - \mathbb{E}[\mu_Y]) + \sigma_Y^2 \\ \mathbb{E}[X] &= \mu_Y^2 + 2\mu_Y(\mu_Y - \mu_Y) + \sigma_Y^2 \\ \mathbb{E}[X] &= \mu_Y^2 + \sigma_Y^2 \end{aligned}$$

Thus, the predicted raw counts from the Gaussian model are obtained by adding the model's residual variance to the squared fitted values. This can then be used to calculate the RMSE in the original count scale from the square root scale predicted values generated by the Gaussian mixed effects model. This is then compared directly to the Poisson and negative-binomial model RMSEs. We can also use this property to forecast the CD4 trajectories in their original scale.

There are several pros and cons to this arrangement. ## Pros The identity-link Gaussian model simplifies the log-posterior greatly, within the log-posterior the random effects are present in their log-priors and also in the log-likelihood, within the log-likelihood they are found in the context of the inverse-link function. For a log-link this would mean nested in an exponential term, this means that the log-posterior will have a highly irregular shape which will result in the Hamiltonian Monte-Carlo taking longer to converge on the parameters with the highest probability density.

Additionally we have successfully solved for the Jensen's inequality issue. We can fit this model and back-transform the predictions to the original scale without any approximations.

Cons

We could overcome the Jensen's inequality issue by modelling the count data directly, in particular the negative-binomial distribution will be able to handle discrete counts while also addressing the heavy tail directly with a parameter that gets fitted to account for the overdispersion in the data.

Another issue with simply square-rooting the CD4 counts is that if this transformation doesn't perfectly normalise the data and remove the heteroskedasticity, the Gaussian distribution is simply misspecified to the data and there is no more that you can do other than fitting a different exponential family distribution model.

In theory the Gaussian distribution is entirely misspecified to the square-root CD4 counts as the Gaussian distribution has a support of $(-\infty, \infty)$ which not only violates the range of the square-root function, but it also violates the fact that CD4 counts are restricted to positive values as they are a physical quantity. The fitted model will almost certainly assign a negligible predicted probability of a patient having a negative square-root CD4 count, however the non-zero aspect implies that this distribution is simply incorrect.

Alternative Proposal

There is another exponential family distribution that might have worked well for the square rooted counts, however there is one glaring issue. The Gamma distribution has a support of $(0, \infty)$ meaning we lose the unrealistic assumption of negative CD4 counts and we also have an additional dispersion parameter to account for any residual heteroskedasticity. The main issue is that the gamma distribution is not defined at 0 therefore the mass of counts at 0 cannot be handled without introducing an artificial offset.

This is a massive barrier and as such I will not attempt to use it, as it would require sensitivity analysis of the model to the offset introduced to see how robust the model actually is.

Survival model

We can infer which survival model would fit best to this data by first fitting a generalised gamma distribution to the survival times.

$$\begin{aligned}\ln(T) &= \mu + \sigma w \\ w &= \frac{\ln(t) - \mu}{\sigma} \\ f(t \mid \mu, \sigma, Q) &= \frac{|Q|(Q^{-2})^{Q-2}}{\sigma t \Gamma(Q^{-2})} \exp [Q^{-2} (Qw - \exp(Qw))]\end{aligned}$$

If we set $Q = 1$ then we get the following.

$$f(t \mid \mu, \sigma, 1) = \frac{|1|(1^{-2})^{1^{-2}}}{\sigma t \Gamma(1^{-2})} \exp [1^{-2} (1w - \exp(1w))] \\ f(t \mid \mu, \sigma) = \frac{1}{\sigma t} \exp [w - \exp(w)]$$

Using the definition of w we get,

$$\frac{dw}{dt} = \frac{d}{dt} \left(\frac{\ln(t) - \mu}{\sigma} \right) = \frac{1}{\sigma t}$$

We can get the pdf of w by using the following identity,

$$f_T(t) = \left| \frac{dw}{dt} \right| f_W(w) = \frac{1}{\sigma t} f_W(w)$$

Therefore,

$$f_W(w) = \exp [w - \exp(w)] \quad (1)$$

$$\implies w \sim \text{Min-EV-Gumbel}(0, 1) \quad (2)$$

We want to get from the minimum extreme value Gumbel distribution to the Weibull, we can do this via the following.

$$e^w = \exp \left(\frac{\ln(t) - \mu}{\sigma} \right) = (e^{\ln t - \mu})^{1/\sigma} = t^{1/\sigma} e^{-\mu/\sigma}.$$

Therefore,

$$f_T(t) = \frac{1}{\sigma t} t^{1/\sigma} e^{-\mu/\sigma} e^{-t^{1/\sigma} e^{-\mu/\sigma}}$$

Now we set $k = 1/\sigma$ and $\lambda = e^\mu$,

$$f_T(t) = k \frac{1}{t} t^k (e^{-\mu})^k e^{-t^k (e^{-\mu})^k} \quad (3)$$

$$= k t^{k-1} \lambda^{-k} e^{-t^k \lambda^{-k}} \quad (4)$$

$$= k t^{k-1} \frac{1}{\lambda \lambda^{k-1}} e^{-(t/\lambda)^k} \quad (5)$$

$$= \frac{k}{\lambda} \left(\frac{t}{\lambda} \right)^{k-1} e^{-(t/\lambda)^k} \quad (6)$$

$$\implies T \sim \text{Weibull}(k, \lambda) \equiv \text{Weibull}(1/\sigma, e^\mu) \quad (7)$$

If we now set $k = 1 \implies 1/\sigma = 1 \implies \sigma = 1$ then we have the following.

$$f_T(t) = \frac{1}{\lambda} e^{-\frac{t}{\lambda}} \quad (8)$$

$$\implies T \sim \text{Exp}(1/\lambda) \quad (9)$$

Therefore $Q = \sigma = 1 \implies T \sim \text{Exp}(1/\lambda)$

Now we can examine what happens when $Q \rightarrow 0$. First I manipulate $f_T(t)$ to evaluate the limit easier.

$$f_T(t) = \frac{|Q|(Q^{-2})^{Q^{-2}}}{\sigma t \Gamma(Q^{-2})} \exp \left[\frac{Qw - \exp(Qw)}{Q^2} \right] \quad (10)$$

$$= \frac{|Q|(Q^{-2})^{Q^{-2}}}{\sigma t \Gamma(Q^{-2})} \exp \left[\frac{-1 + 1 + Qw - \exp(Qw)}{Q^2} \right] \quad (11)$$

$$= \frac{|Q|(Q^{-2})^{Q^{-2}}}{\sigma t \Gamma(Q^{-2})} e^{-1/Q^2} \exp \left[\frac{1 + Qw - \exp(Qw)}{Q^2} \right] \quad (12)$$

$$= \frac{|Q|(Q^{-2})^{Q^{-2}}}{\sigma t \Gamma(Q^{-2}) e^{Q^{-2}}} \exp \left[\frac{1 + Qw - \exp(Qw)}{Q^2} \right] \quad (13)$$

$$= C(Q) e^{A(Q)} \quad (14)$$

Now we take $\lim_{Q \rightarrow 0} f_T(t)$, to find this limit we need to find $\lim_{Q \rightarrow 0} C(Q)$ and $\lim_{Q \rightarrow 0} A(Q)$.

$\lim_{Q \rightarrow 0} C(Q)$:

$$\text{Let } k = Q^{-2} \text{ therefore } C(k) = \frac{k^k |1/\sqrt{k}|}{\sigma t \Gamma(k) e^k} = \frac{k^{k-1/2}}{\sigma t \Gamma(k) e^k}$$

We know that as $Q \rightarrow 0$, $k = 1/Q^2 \rightarrow \infty$. Stirling's approximation is an asymptotic relationship for large values of the gamma function, since $k \rightarrow \infty$ it becomes an equivalence.

Stirling's approximation tells us that,

$$\lim_{k \rightarrow \infty} \Gamma(k) = \lim_{k \rightarrow \infty} \sqrt{\frac{2\pi}{k}} \left(\frac{k}{e} \right)^k = \lim_{k \rightarrow \infty} \sqrt{2\pi} k^{k-1/2} e^{-k}$$

We can now plug this in to demonstrate the following,

$$\lim_{Q \rightarrow 0} C(Q) = \lim_{k \rightarrow \infty} C(k) = \lim_{k \rightarrow \infty} \frac{k^{k-1/2}}{\sigma t \Gamma(k) e^k} = \lim_{k \rightarrow \infty} \frac{k^{k-1/2}}{\sigma t \sqrt{2\pi} k^{k-1/2} e^{-k} e^k} = \lim_{k \rightarrow \infty} \frac{1}{\sigma t \sqrt{2\pi}} = \frac{1}{\sigma t \sqrt{2\pi}}$$

$\lim_{Q \rightarrow 0} A(Q)$: By substituting $Q = 0$ into $A(Q)$ we get the following $\frac{1+0w-\exp(0w)}{0^2} = \frac{0}{0}$ which is an indeterminate form therefore we can use L'Hôpital's rule to evaluate the limit.

$$\lim_{Q \rightarrow 0} A(Q) = \lim_{Q \rightarrow 0} \left(\frac{\frac{d}{dQ} (1 + Qw - e^{Qw})}{\frac{d}{dQ} (Q^2)} \right) = \lim_{Q \rightarrow 0} \left(\frac{w - we^{Qw}}{2Q} \right) = \frac{0}{0}$$

This again is in an indeterminate form therefore we apply L'Hôpital's rule again to get the following,

$$\lim_{Q \rightarrow 0} A(Q) = \lim_{Q \rightarrow 0} \left(\frac{\frac{d}{dQ} (w - we^{Qw})}{\frac{d}{dQ} (2Q)} \right) = \lim_{Q \rightarrow 0} \left(\frac{-w^2 e^{Qw}}{2} \right) = -w^2/2$$

We have that $\lim_{Q \rightarrow 0} C(Q)$ and $\lim_{Q \rightarrow 0} A(Q)$ exist therefore,

$$\lim_{Q \rightarrow 0} f_T(t) = \lim_{Q \rightarrow 0} C(Q) e^{A(Q)} \quad (15)$$

$$= \left[\lim_{Q \rightarrow 0} C(Q) \right] e^{[\lim_{Q \rightarrow 0} A(Q)]} \quad (16)$$

$$= \frac{1}{\sigma t \sqrt{2\pi}} e^{-w^2/2} = \left| \frac{dw}{dt} \right| f_W(w) = \frac{1}{\sigma t} f_W(w) \quad (17)$$

This means that we have that $f_W(w) = \frac{1}{\sqrt{2\pi}}e^{-w^2/2} \implies w \sim N(0, 1)$ therefore from $\ln(T) = \mu + \sigma w$ we can see that $\ln(T) \sim N(\mu, \sigma^2)$ meaning that $T \sim \text{Log-Normal}(\mu, \sigma^2)$

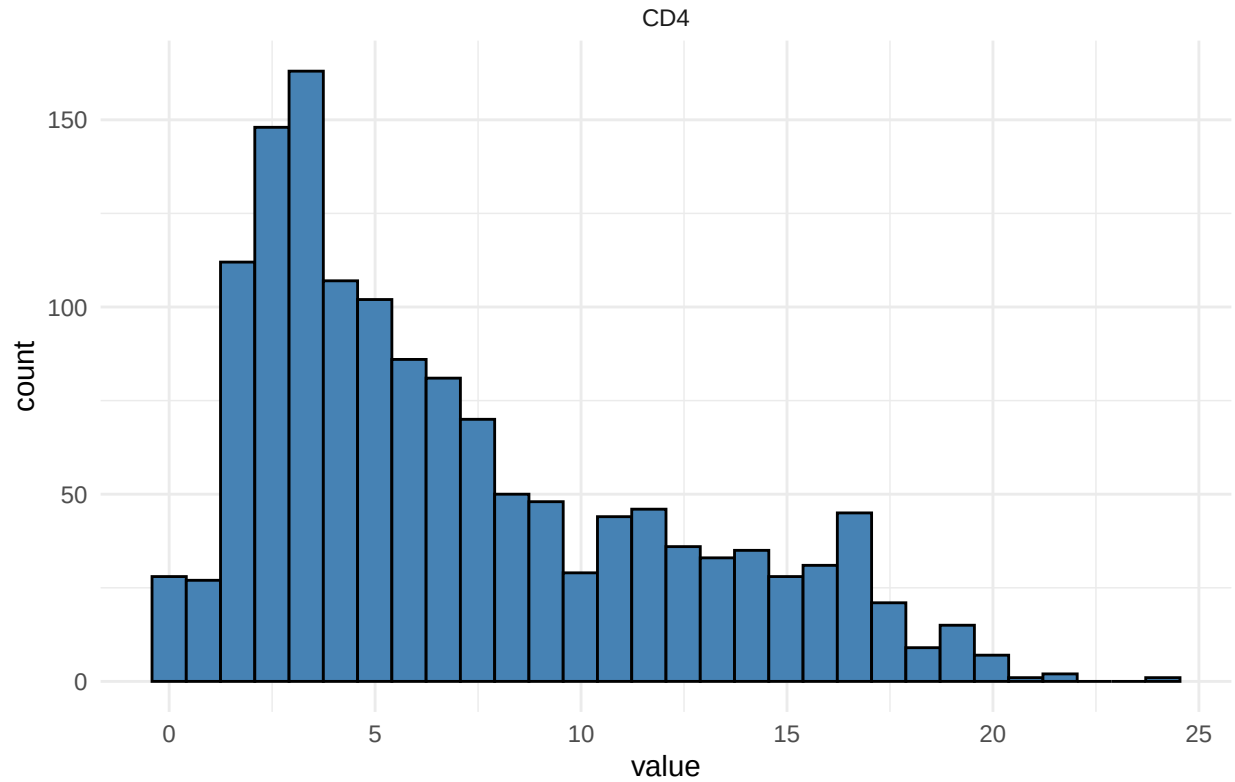
There is one more model nested within the generalised-gamma framework, when we choose $Q = \sigma$ we get the following decomposition

```
generalised_gamma_ph <- flexsurvreg(Surv(Time, death) ~ drug + gender + CD4,
                                   data = aids.id,
                                   dist = "gengamma")
tidy(generalised_gamma_ph)
```

```
## # A tibble: 6 x 5
##   term      estimate std.error statistic  p.value
##   <chr>      <dbl>    <dbl>    <dbl>    <dbl>
## 1 mu         2.28      0.221      NA      NA
## 2 sigma      0.775      0.113      NA      NA
## 3 Q          0.777      0.260      NA      NA
## 4 drugddI    -0.184      0.106     -1.74    8.26e- 2
## 5 gendermale  0.194      0.183      1.06    2.88e- 1
## 6 CD4        0.129      0.0168     7.68    1.60e-14
```

```
aids %>%
  select(CD4) %>%
  pivot_longer(everything()) %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "black") +
  facet_wrap(~name, scales = "free") +
  theme_minimal() +
  labs(title = "Biomarker Distribution")
```

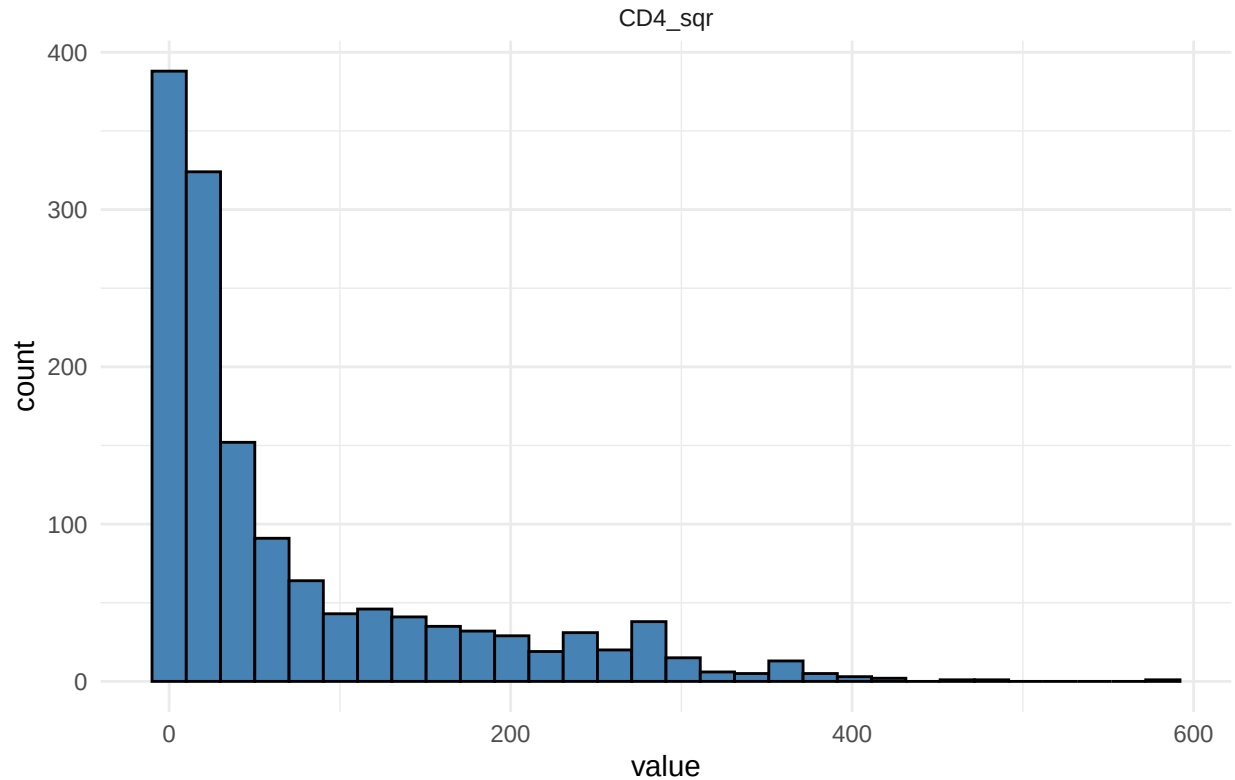
Biomarker Distribution



```
aids$CD4_sqr <- round((aids$CD4)**2)

aids %>%
  select(CD4_sqr) %>%
  pivot_longer(everything()) %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "black") +
  facet_wrap(~name, scales = "free") +
  theme_minimal() +
  labs(title = "Biomarker Distribution")
```

Biomarker Distribution



```
negbinom_CD4 <- mixed_model(CD4_sqr ~ obstime + obstime:drug,
                             random = ~ obstime | patient,
                             data = aids,
                             family = negative.binomial)
summary(negbinom_CD4)
```

```
##
## Call:
## GLMMadaptive::mixed_model(fixed = CD4_sqr ~ obstime + obstime:drug,
##   random = ~obstime | patient, data = aids, family = negative.binomial)
##
## Data Descriptives:
## Number of Observations: 1405
## Number of Groups: 467
##
## Model:
## family: negative binomial
## link: log
##
## Fit statistics:
##   log.Lik    AIC    BIC
## -6696.818 13407.64 13436.66
##
## Random effects covariance matrix:
##           StdDev   Corr
## (Intercept) 1.3395
```

```

## obstime      0.0617  0.1560
##
## Fixed effects:
##              Estimate Std.Err z-value p-value
## (Intercept)    3.5988  0.0654 55.0402 < 1e-04
## obstime       -0.0683  0.0073 -9.3479 < 1e-04
## obstime:drugddI  0.0133  0.0101  1.3219 0.18619
##
## log(dispersion) parameter:
##   Estimate Std.Err
##    1.6013  0.0725
##
## Integration:
## method: adaptive Gauss-Hermite quadrature rule
## quadrature points: 11
##
## Optimization:
## method: hybrid EM and quasi-Newton
## converged: TRUE

poisson_CD4 <- mixed_model(CD4_sqr ~ obstime + obstime:drug,
                           random = ~ obstime | patient,
                           data = aids,
                           family = poisson(link = log))
summary(poisson_CD4)

##
## Call:
## GLMMadaptive::mixed_model(fixed = CD4_sqr ~ obstime + obstime:drug,
##   random = ~obstime | patient, data = aids, family = poisson(link = log))
##
## Data Descriptives:
## Number of Observations: 1405
## Number of Groups: 467
##
## Model:
## family: poisson
## link: log
##
## Fit statistics:
##   log.Lik      AIC      BIC
## -9021.988 18055.98 18080.85
##
## Random effects covariance matrix:
##              StdDev   Corr
## (Intercept)  1.7606
## obstime      0.1552  0.1941
##
## Fixed effects:
##              Estimate Std.Err z-value p-value
## (Intercept)    4.5810  0.3265 14.0315 < 1e-04
## obstime       -0.0142  0.0266 -0.5335 0.59367
## obstime:drugddI  0.0130  0.0168  0.7720 0.44013
##
## Integration:

```

```

## method: adaptive Gauss-Hermite quadrature rule
## quadrature points: 11
##
## Optimization:
## method: hybrid EM and quasi-Newton
## converged: TRUE

cox_aids2 <- coxph(Surv(Time, death) ~ drug + gender, data = aids.id)

summary(cox_aids2)

## Call:
## survival::coxph(formula = Surv(Time, death) ~ drug + gender,
##   data = aids.id)
##
##   n= 467, number of events= 188
##
##               coef exp(coef) se(coef)      z Pr(>|z|)
## drugddI      0.2116   1.2357   0.1462  1.447   0.148
## gendermale -0.1708   0.8430   0.2421 -0.705   0.481
##
##               exp(coef) exp(-coef) lower .95 upper .95
## drugddI      1.236      0.8093   0.9277   1.646
## gendermale    0.843      1.1862   0.5245   1.355
##
## Concordance= 0.536 (se = 0.02 )
## Likelihood ratio test= 2.55 on 2 df,  p=0.3
## Wald test               = 2.56 on 2 df,  p=0.3
## Score (logrank) test = 2.57 on 2 df,  p=0.3

joint_fit2 <- jm(cox_aids2, negbinom_CD4, time_var = "obstime", n_iter = 15000, n_burnin = 2500)

summary(joint_fit2)

##
## Call:
## JMBayes2::jm(Surv_object = cox_aids2, Mixed_objects = negbinom_CD4,
##   time_var = "obstime", n_iter = 15000, n_burnin = 2500)
##
## Data Descriptives:
## Number of Groups: 467           Number of events: 188 (40.3%)
## Number of Observations:
##   CD4_sqr: 1405
##
##               DIC      WAIC      LPML
## marginal    14993.60 14970.67 -7488.711
## conditional 14562.16 14303.05 -7567.084
##
## Random-effects covariance matrix:
##
##           StdDev   Corr
## (Intr) 1.3518 (Intr)
## obstim 0.0639 0.2152
##
## Survival Outcome:

```

```

##               Mean StDev   2.5%   97.5%     P   Rhat
## drugddI       0.3454 0.1965 -0.0387  0.7275 0.0773 1.0011
## gendermale   -0.2227 0.2677 -0.7213  0.3322 0.3948 1.0015
## value(CD4_sqr) -0.5845 0.0632 -0.7098 -0.4635 0.0000 1.0018
##
## Longitudinal Outcome: CD4_sqr (family = negative binomial, link = log)
##               Mean StDev   2.5%   97.5%     P   Rhat
## (Intercept)    3.5979 0.0663  3.4683  3.7274 0.0000 1.0001
## obstime       -0.0743 0.0076 -0.0893 -0.0597 0.0000 1.0002
## obstime:drugddI 0.0094 0.0101 -0.0101  0.0293 0.3491 1.0005
## sigma         5.0172 0.3707  4.3386  5.7867 0.0000 1.0011
##
## MCMC summary:
## chains: 3
## iterations per chain: 15000
## burn-in per chain: 2500
## thinning: 1
## time: 2.3 min

# 1. Randomly select exactly 4 patients to match the 2x2 grid layout
set.seed(129)
random_patients <- sample(aids.id$patient, 4)

# 2. Set up the graphics window for a 2x2 grid with adjusted margins
par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))

# 3. Loop through the 4 patients and generate both longitudinal and survival panels
for (pid in random_patients) {

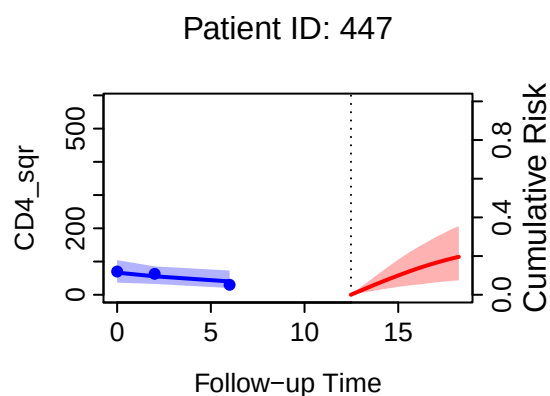
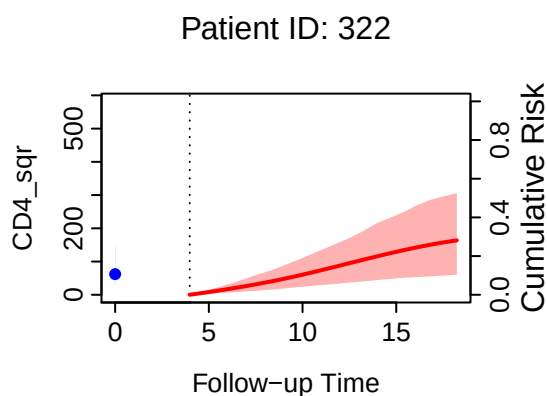
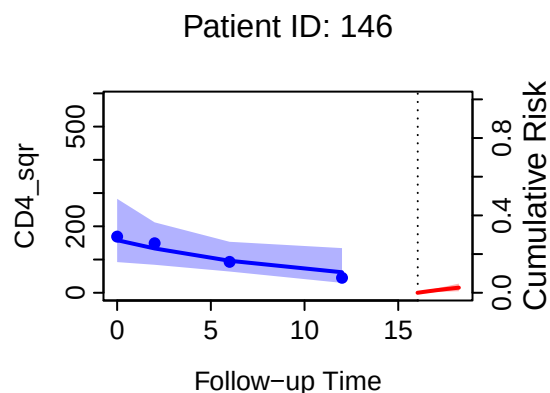
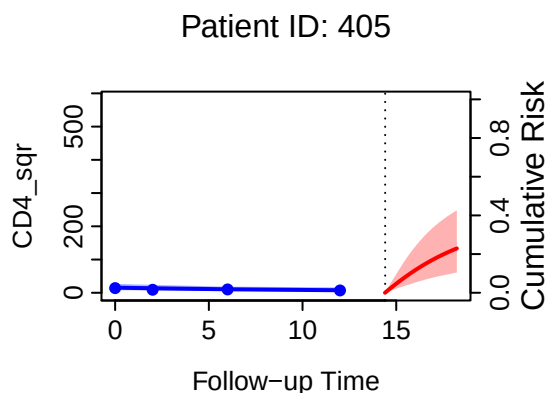
  # Extract this specific patient's chart
  pt_data <- aids[aids$patient == pid, ]

  # Forecast their CD4 count trajectory
  pred_long <- predict(joint_fit2, newdata = pt_data, return_newdata = TRUE)

  # Forecast their survival/event risk
  pred_event <- predict(joint_fit2, newdata = pt_data, process = "event", return_newdata = TRUE)

  # Plot the dual-panel output with explicit Patient ID titles
  plot(pred_long, pred_event,
       main = paste("Patient ID:", pid),
       xlim = c(0, 25))
}

```



4. Reset the plotting window back to normal when finished

```
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2))
```

Calculate the Time-Dependent ROC curve

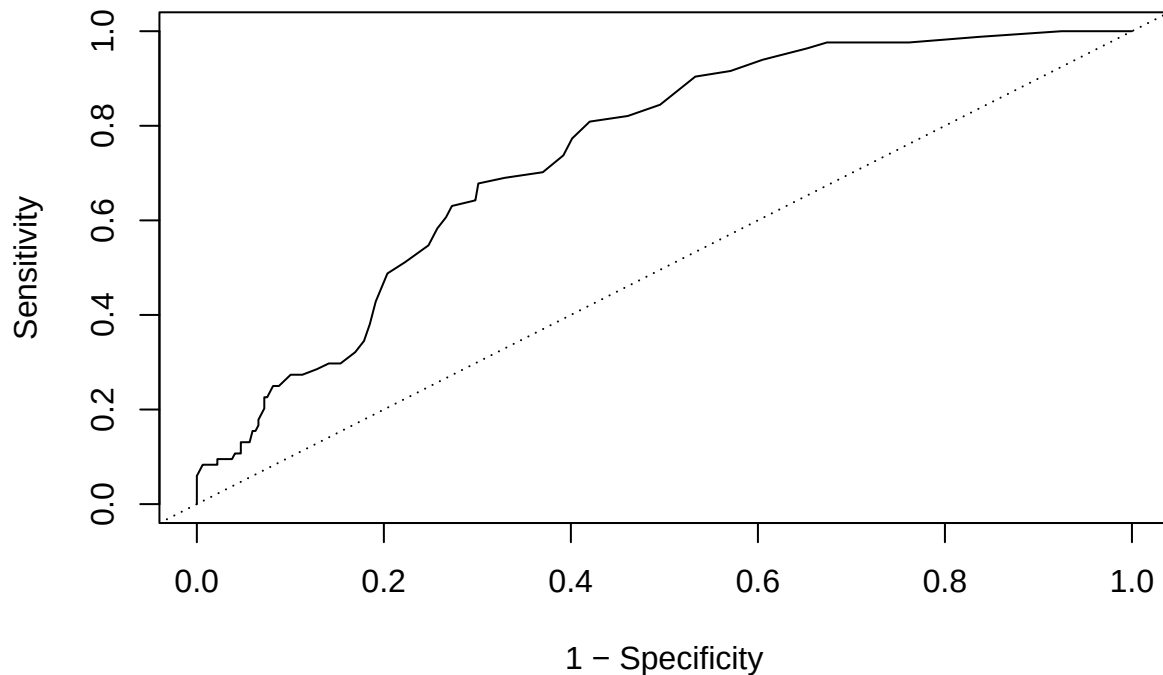
```
roc_quiz <- tvROC(joint_fit2, newdata = aids, Tstart = 6, Dt = 6, cores = 1)
```

Print the Area Under the Curve (AUC) score

```
summary(roc_quiz)
```

```
##           Length Class  Mode
## TP           101  -none- numeric
## FP           101  -none- numeric
## nTP           101  -none- numeric
## nFN           101  -none- numeric
## nFP           101  -none- numeric
## nTN           101  -none- numeric
## tp          20200  -none- numeric
## fp          20200  -none- numeric
## thrs          101  -none- numeric
## F1score         1  -none- numeric
## Youden           1  -none- numeric
## Tstart           1  -none- numeric
## Thoriz           1  -none- numeric
## nr               1  -none- numeric
## classObject      1  -none- character
## type_weights      1  -none- character
## nameObject        1  -none- character
```

```
# Plot the ROC curve
plot(roc_quiz)
```



```
tvAUC(joint_fit2, newdata = aids, Tstart = 6, Dt = 6, cores = 1)
```

```
##
## Time-dependent AUC for the Joint Model joint_fit2
##
## Estimated AUC: 0.7389
## At time: 12
## Using information up to time: 6 (403 subjects still at risk)
## Accounting for censoring using model-based weights
# Calculate the Brier Score to check prediction accuracy
brier_quiz <- tvBrier(joint_fit2, newdata = aids, Tstart = 6, Dt = 6)

# Print the result
brier_quiz

##
## Prediction Error for the Joint Model 'joint_fit2'
##
## Estimated Brier score: 0.1492
## At time: 12
## For the 403 subjects at risk at time 6
## Number of subjects with an event in [6, 12): 84
## Number of subjects with a censored time in [6, 12): 1
```

```

## Accounting for censoring using model-based weights
set.seed(123)
unique_ids <- unique(aids$patient)
n_folds <- 5

# Create a balanced, randomized vector of fold assignments (1 through 5)
folds <- sample(rep(1:n_folds, length.out = length(unique_ids)))

cv_auc <- numeric(n_folds)
cv_brier <- numeric(n_folds)

# 2. Run the actual refitting loop
for (i in 1:n_folds) {
  cat("Running Fold", i, "of", n_folds, "...\\n")

  # Identify train and test IDs
  test_ids <- unique_ids[folds == i]
  train_ids <- unique_ids[folds != i]

  # Split longitudinal and survival datasets strictly by patient ID
  train_data <- aids[aids$patient %in% train_ids, ]
  train_data.id <- aids.id[aids.id$patient %in% train_ids, ]
  test_data <- aids[aids$patient %in% test_ids, ]

  # Refit the strictly correct Negative Binomial using Gauss-Kronrod quadrature
  nb_fit <- mixed_model(CD4_sqr ~ obstime + obstime:drug,
                        random = ~ obstime | patient,
                        data = train_data,
                        family = negative.binomial)

  # Refit the survival submodel
  cox_fit <- coxph(Surv(Time, death) ~ drug + gender, data = train_data.id)

  # Refit the joint model
  fold_jm <- jm(cox_fit, nb_fit, time_var = "obstime", n_iter = 8000, n_burnin = 1300)
  summary(fold_jm)

  # Test on the remaining 20% unseen data (cores = 1 to prevent socket crashes)
  cv_auc[i] <- tvAUC(fold_jm, newdata = test_data, Tstart = 6, Dt = 6, cores = 1)$auc
  cv_brier[i] <- tvBrier(fold_jm, newdata = test_data, Tstart = 6, Dt = 6, cores = 1)$Brier
}

## Running Fold 1 of 5 ...
## Running Fold 2 of 5 ...
## Running Fold 3 of 5 ...
## Running Fold 4 of 5 ...
## Running Fold 5 of 5 ...

# 3. See the true, un-cheated cross-validation scores
cat("\\n5-Fold CV AUC:", mean(cv_auc, na.rm = TRUE), "\\n")

##
## 5-Fold CV AUC: 0.7418798

```

```
cat("5-Fold CV Brier:", mean(cv_brier, na.rm = TRUE), "\n")
```

```
## 5-Fold CV Brier: 0.1512043
```